



School of Computer Science

COMP 8347 “Internet Applications and Distributed
Systems”

Summer 2026

Week 9 - I

Distributed Systems: Scalability & the CAP Theorem

DR. OLENA SYROTKINA



University of Windsor

Copyright Notice

This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero**, **Chegg**, **StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.

Agenda

- What is a Distributed System?
- Vertical vs. Horizontal Scaling
- Replication (Copies of Data)
- Consistency vs. Availability
- The CAP Theorem
- CP Systems (Consistency + Partition Tolerance)
- AP Systems (Availability + Partition Tolerance)
- Real-World Examples

What is a Distributed System?

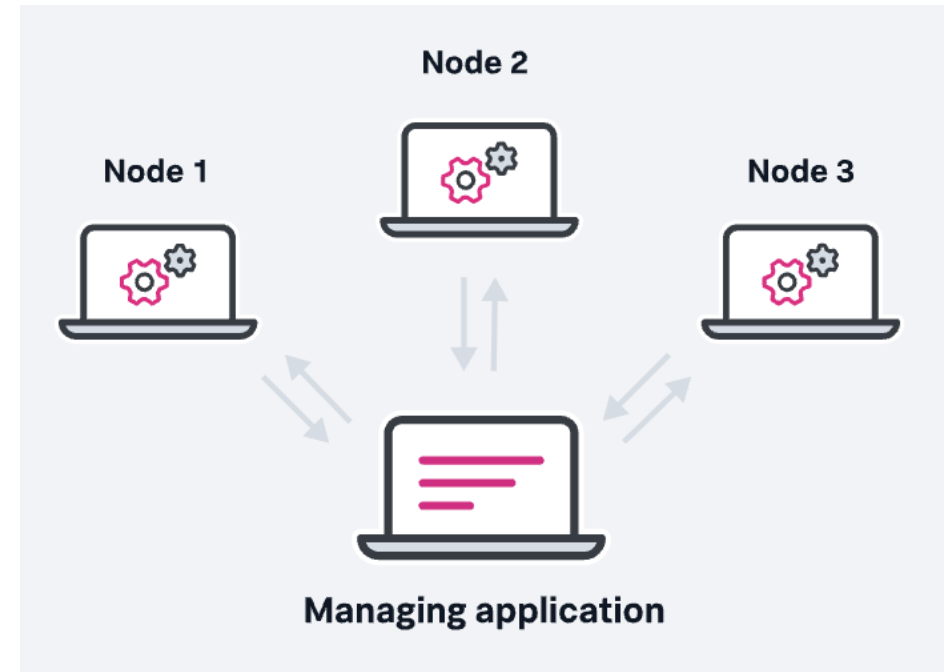
A **distributed system** is when your application runs on multiple computers (servers/nodes) that work together. Users should feel like it's "one system," but under the hood it's not.

Typical reasons to distribute:

- ✓ Your single server is too slow
- ✓ You need more storage
- ✓ You need higher fault-tolerance
- ✓ You want global performance (users around the world)

We call these machines **nodes**.

Nodes need to *coordinate and communicate*.



Vertical vs. Horizontal Scaling

Vertical Scaling (Scale Up)

Vertical scaling means you **keep the same machine**, but you **upgrade its hardware** so it can handle more work.

- ✓ You don't change your application architecture.
- ✓ You just make one server stronger.

How you scale up:

- ✓ Add more **CPU cores** (faster processing)
- ✓ Add more **RAM** (can hold more data in memory)
- ✓ Add faster **storage** (SSD → NVMe → persistent RAM, etc.)

Benefit	Why it matters
Simple to implement	No new machines to configure.
No software architecture changes	The system still behaves like one server.
Fast short-term performance improvement	Great for quick fixes.

Vertical vs. Horizontal Scaling

Limitation	Why it matters
There is a physical limit	You cannot just add unlimited RAM/CPU; hardware maxes out.
Can get very expensive	High-end server hardware scales non-linearly in price.
Single point of failure remains	If that one server goes down, your whole system is down.

When Vertical Scaling Works Well

Situation	Why it fits
Fast temporary fix needed	Cheap in terms of developer time.
Application is not built for distributed systems	Avoids rewriting your system.
You are at early stages of growth	Easier to start small.

When Vertical Scaling Becomes a Problem

- When **you keep needing more power** every few months
- When **downtime becomes costly**
- When performance is **limited by hardware max specs**

That's when organizations start considering **Horizontal Scaling (Scale Out)**.

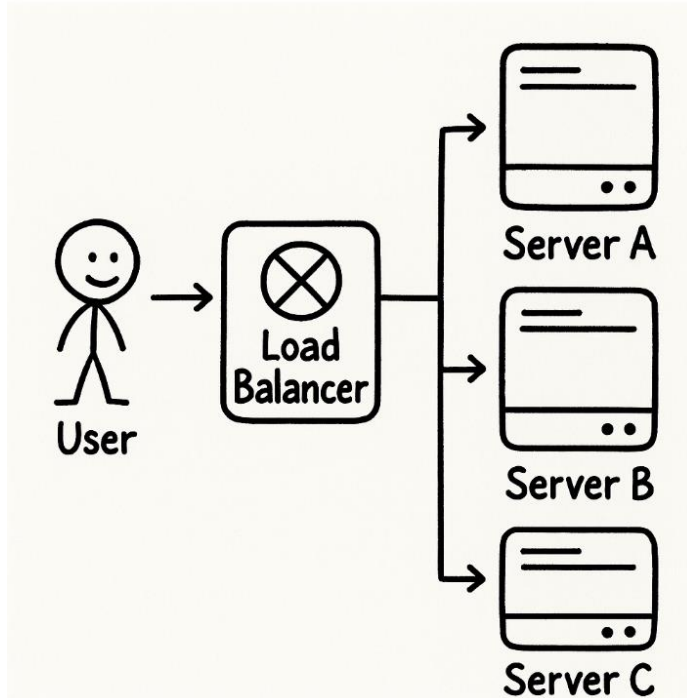
Vertical vs. Horizontal Scaling

Horizontal Scaling (Scale Out)

Instead of upgrading **one machine**, you **add more machines** and have them work together.

- ✓ You now have multiple servers (nodes) sharing the load.
- ✓ The application must be able to run on multiple nodes, and requests must be distributed among them.

How it Works in a System



The **Load Balancer** decides which server handles each request.

Benefit	Why it matters
High scalability	You can keep adding machines as traffic grows.
Fault-tolerant	If one server fails, the system stays up.
Better for global performance	Nodes can be placed closer to users around the world.

Vertical vs. Horizontal Scaling

Limitation	Why it matters
Requires architecture changes	App must support running across multiple servers.
Requires coordination	Sessions, databases, and caches must be shared.
More operational complexity	You need load balancing, monitoring, auto-scaling, etc.

When Horizontal Scaling Works Well

Scenario	Reason
Large, growing user base	Adding servers is easy and predictable.
Need high availability	No single failure brings system down.
Serving users globally	You can run servers in multiple regions.

When Horizontal Scaling is Hard

- Applications that **store state in memory** need refactoring
- Databases must be shared or replicated
- Caching layers need to be centralized (Redis, Memcached)

Vertical vs. Horizontal Scaling

Feature	Vertical (Scale Up)	Horizontal (Scale Out)
Main strategy	Make one machine stronger	Add more machines
Complexity	Simple	More complex
Cost	Hardware becomes expensive	Cheaper hardware but more of it
Max capacity	Limited by hardware	Can scale almost indefinitely
Fault tolerance	Single point of failure	Highly fault-tolerant

Question:

Your app is slowing down. You can either:

A) Upgrade the server to 128 GB RAM

B) Add 3 cheaper servers and load balance

Which one is better long-term, and why?

Replication (Copies of Data)

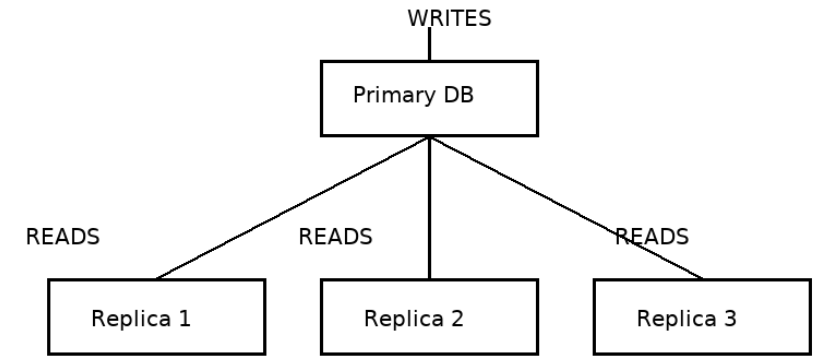
Replication means **you keep multiple copies of the same data** on different servers (nodes). These nodes work together so that data stays **accessible, fast, and safe** even if failures happen.

Why Replicate Data?

Reason	Explanation
Availability	If one server goes down, another one still has the data.
Fault Tolerance	Hardware fails. Replication ensures the system keeps working.
Read Performance	Users can read from the closest or least-busy copy.
Global Access	Copies can be placed in different geographic regions to reduce latency.

E.g. You have a database.

To improve reliability and read performance, you replicate it like this:



- The **Primary** handles all **writes** (updates, inserts).
- The **Replicas** handle **reads**, so traffic is shared.

This is called **Primary-Replica Replication** (or Leader-Follower).

Replication (Copies of Data)

Benefit	Why it matters
High availability	If primary goes down, a replica can take over.
Improved read scalability	Many users can read simultaneously.
Better geographic performance	Replicas near users = faster response.

Problem	Why it's tricky
Replication Lag	Replicas may receive updates slightly later than the primary.
More complexity	You need monitoring, failover strategy, and sync logic.
Write scaling is still limited	Writes still go to one primary machine (unless using more advanced approaches).

When Replication Helps Most

- Apps with **many more reads than writes**

Example: News sites, blogs, dashboards, e-commerce product pages

- Apps where **uptime is critical**
- Globally-distributed users (US, Europe, Asia)

Consistency vs. Availability

When you replicate data across multiple servers, you face a fundamental question:

1. **Do all users always see the same data (Consistency)?**
2. **Or is the system always up and responsive even if some servers fail (Availability)?**

You cannot maximize both at the same time in real distributed systems.

This is the core idea behind the **CAP Theorem**

Consistency

Consistency means that all nodes see the **same data** at the same time. If one user updates a record, **everyone immediately sees the new value**, no matter which server they are talking to.

Example:

You change your profile picture.

Consistent system behavior:

- You refresh the page
- Your new photo appears everywhere instantly

But to do this:

- All servers must coordinate
- Servers may need to wait for each other
- If one server is unreachable, the system may **delay or reject requests**

Therefore, consistency can reduce availability.

Consistency vs. Availability

Availability

Availability means that the system always responds, even if some servers are down or disconnected.

The downside:

Some nodes may still have **old data** (because updates haven't reached them yet)

Users might not see the very latest value for a short time

Example:

1. You change your profile picture.
2. You refresh.
3. One server hasn't gotten the update yet.

Therefore....

1. Some users see the new photo
2. Some still see the old one for a moment
3. But the system **never goes down**.
4. Availability can reduce consistency.

Consistency vs. Availability

This Leads to a Trade-Off

Goal	What it Requires	What You Give Up
Consistency	Servers wait for each other and update synchronously	Some requests may fail or wait if a server is down
Availability	Servers respond instantly with whatever data they have	Some responses may show slightly outdated data

Real-World Examples

System	Prioritizes	Why
Bank account systems	Consistency	Balances must never disagree
Instagram feed / TikTok feed	Availability	Users don't care if they see posts 5 seconds late
E-commerce product views	Availability	Showing product info must always work
E-commerce checkout / payment	Consistency	Must prevent double-charging and negative inventory

The CAP Theorem (Eric Brewer, 2000)

The **CAP Theorem** states that in a distributed system, you can only guarantee **two** of the following **three** properties at the same time:

Letter	Meaning
C	Consistency (every node returns the same data)
A	Availability (system always responds, even during failures)
P	Partition Tolerance (system continues working even if communication between nodes fails)

What is Partition Tolerance?

A **partition** happens when nodes in the system **can't talk to each other** due to:

- Network failure
- Data center outage
- Fiber cable cut
- Node reboot

Partition tolerance means:

The system **keeps running**, even though messages are delayed or dropped. In the real world, **network failures are inevitable**, so we must have **P**. This leaves us choosing **between C and A** during failures.

The CAP Theorem (Eric Brewer, 2000)

Therefore, During a Network Partition, You Pick One:

Choice	Outcome
CP System (Consistency + Partition Tolerance)	System may pause or reject requests to keep data consistent.
AP System (Availability + Partition Tolerance)	System stays up but may return old data temporarily.

Examples of CAP Choices in Real Systems

System / Service	Type	Why
Traditional SQL Databases (Postgres, MySQL Cluster, etc.)	CP	They stop writes when consistency would break.
NoSQL Databases (Cassandra, DynamoDB, Riak)	AP	They allow reads/writes even when some replicas are behind.
Banking / Payments	CP	Must prevent incorrect balances.
Social Feeds (Instagram, Twitter, TikTok)	AP	Slightly stale feed is acceptable.

CP Systems (Consistency + Partition Tolerance)

These systems **prefer correctness over availability**.

If the network is unreliable, or replicas disagree, the system may:

- ✓ **Reject requests**
- ✓ **Pause writes**
- ✓ **Return errors**

Rather than return incorrect data.

Example System	Why CP?	Real Use Case
PostgreSQL, MySQL (single primary)	Ensures all writes are consistent and durable	Banking systems, inventory, financial ledgers
MongoDB in “majority write” mode	Writes succeed only if most nodes confirm	Enterprise transactional systems
Zookeeper / etcd / Consul	Used for configuration & coordination – must always be correct	Service discovery, distributed locks

CP Systems (Consistency + Partition Tolerance)

When You Choose CP:

- ✓ **Money** is involved (banking, crypto, balance transfers)
- ✓ **State must be exact** (voting systems, ordering systems)
- ✓ **Incorrect data is worse than no data**

Key point:

“CP = I'd rather be correct than fast.”

AP Systems (Availability + Partition Tolerance)

These systems **prefer being online** over guaranteeing perfectly synchronized data.
If some replicas are behind, the system **still responds**, possibly with old data.

Example System	Why AP?	Real Use Case
Cassandra	Designed for always-on global deployments	Messaging, real-time analytics
Amazon DynamoDB	Sacrifices immediate consistency for uptime	Shopping carts at massive scale
Riak, CouchDB	“Eventually consistent” by default	Offline-first mobile apps, content delivery systems

When You Choose AP:

- System must **never go down**
- Data can be **eventually synchronized**
- **Stale data is acceptable for a short time**

Examples:

- Instagram feed
- TikTok feed
- Product catalog viewing

Key point:

“AP = I’d rather be available than perfect.”

Real-World Examples

Shared Photo Albums (iCloud / Google Photos / WhatsApp)

Imagine you and your friends are on a trip and you're all adding photos to the **same shared album**. The album is stored on **multiple servers** around the world, so people everywhere can load it fast. Now consider what happens depending on how the platform is designed:

Case 1: Consistency Priority (CP)

“Everyone must always see the newest photos immediately.”

- When someone uploads a photo, **all servers must sync** before anyone can see it.
- If one region's data center is temporarily offline, **the app may pause or say “Please wait...”**.
- Everything stays *perfectly consistent*, but:
 - Uploads might feel slow
 - The app may refuse to show the album until sync completes

Experience:

Slow, but always correct.

Feels like:

iCloud sometimes when it says
“*Syncing...*” for a while.

Real-World Examples

Shared Photo Albums (iCloud / Google Photos / WhatsApp)

Imagine you and your friends are on a trip and you're all adding photos to the **same shared album**. The album is stored on **multiple servers** around the world, so people everywhere can load it fast. Now consider what happens depending on how the platform is designed:

Case 2: Availability Priority (AP)

“The album must always load, even if some servers are behind.”

Everyone sees the album right away.

But some friends may see **new photos a few seconds later** than others.

The system stays fast and always works.

Consistency catches up *eventually*.

Experience:

Fast, but sometimes briefly inconsistent.

Feels like:

WhatsApp or Google Photos, where new images sometimes appear a moment later.